

TP n°4 : Introduction au Procedural Language (PL/SQL) d'Oracle

Partie I

Amjad Douiri

13 avril 2026

Exercice n°1

Question 1 — Somme de deux entiers

Écrire une procédure anonyme PL/SQL qui permet de demander à un utilisateur de saisir deux entiers et d'afficher leur somme.

```
1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_a NUMBER;
4     v_b NUMBER;
5     v_somme NUMBER;
6 BEGIN
7     v_a := &entier1;
8     v_b := &entier2;
9     v_somme := v_a + v_b;
10    DBMS_OUTPUT.PUT_LINE('La somme de ' || v_a || ' et '
11                          || v_b || ' est : ' || v_somme);
12 END;
13 /
```

Listing 1 – Somme de deux entiers

Explications :

- La notation `&variable` permet de demander à l'utilisateur de saisir une valeur via SQL*Plus ou SQL Developer (variable de substitution).
- `DBMS_OUTPUT.PUT_LINE` affiche le résultat dans la sortie SGBD.

Question 2 — Table de multiplication

Écrire une procédure anonyme PL/SQL qui permet de demander à un utilisateur de saisir un nombre et d'afficher sa table de multiplication.

```
1 SET SERVEROUTPUT ON;
2 DECLARE
```

```

3      v_nombre NUMBER;
4  BEGIN
5      v_nombre := &nombre;
6      DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' ||
v_nombre || ' :');
7      FOR i IN 1..10 LOOP
8          DBMS_OUTPUT.PUT_LINE(v_nombre || ' x ' || i || ' = '
9              || (v_nombre * i));
10     END LOOP;
11 END;
12 /

```

Listing 2 – Table de multiplication

Explications :

- La boucle FOR i IN 1..10 itère de 1 à 10.
- À chaque itération, on affiche le produit `v_nombre * i`.

Question 3 — Fonction récursive x^n

Écrire une fonction récursive qui permet de retourner x^n , où x et n sont deux entiers positifs.

```

1  SET SERVEROUTPUT ON;
2  -- Création de la fonction récursive
3  CREATE OR REPLACE FUNCTION puissance(x NUMBER, n NUMBER)
4  RETURN NUMBER
5  IS
6  BEGIN
7      IF n = 0 THEN
8          RETURN 1;
9      ELSE
10         RETURN x * puissance(x, n - 1);
11     END IF;
12 END;
13 /
14
15 -- Test de la fonction
16 DECLARE
17     v_x NUMBER := &x;
18     v_n NUMBER := &n;
19     v_resultat NUMBER;
20 BEGIN
21     v_resultat := puissance(v_x, v_n);
22     DBMS_OUTPUT.PUT_LINE(v_x || '^' || v_n || ' = ' ||
v_resultat);
23 END;
24 /

```

Listing 3 – Fonction récursive puissance x^n **Explications :**

- **Cas de base** : si $n = 0$, on retourne 1 (car $x^0 = 1$).
- **Cas récursif** : on retourne $x \times x^{n-1}$, soit `x * puissance(x, n-1)`.
- La fonction est créée avec `CREATE OR REPLACE FUNCTION` pour pouvoir être réutilisée.

Question 4 — Factorielle stockée dans une table

Écrire une procédure anonyme PL/SQL qui calcule la factorielle d'un nombre strictement positif saisi par l'utilisateur. Le résultat sera stocké dans une table `resultatFactoriel`.

```

1  -- Création de la table de résultats
2  CREATE TABLE resultatFactoriel (
3      nombre          NUMBER(10) NOT NULL,
4      factorielle     NUMBER(38) NOT NULL,
5      CONSTRAINT pk_fact PRIMARY KEY (nombre)
6  );
7
8  -- Bloc anonyme de calcul
9  SET SERVEROUTPUT ON;
10 DECLARE
11     v_nombre NUMBER;
12     v_fact   NUMBER := 1;
13 BEGIN
14     v_nombre := &nombre;
15
16     IF v_nombre <= 0 THEN
17         DBMS_OUTPUT.PUT_LINE('Erreur : le nombre doit etre'
18                               || ' strictement positif.');

```

Listing 4 – Factorielle avec stockage dans une table

Explications :

- On commence par créer la table `resultatFactoriel` avec deux colonnes : `nombre` et `factorielle`.
- Le calcul se fait itérativement avec une boucle `FOR`.
- Le résultat est inséré dans la table puis validé avec `COMMIT`.

Question 5 — Factorielles des 20 premiers entiers

Modifier le programme précédent pour qu'il calcule et stocke dans une table `resultatsFactoriels` les factorielles des 20 premiers nombres entiers.

```
1  -- Création de la table
2  CREATE TABLE resultatsFactoriels (
3      nombre          NUMBER(10) NOT NULL,
4      factorielle      NUMBER(38) NOT NULL,
5      CONSTRAINT pk_facts PRIMARY KEY (nombre)
6  );
7
8  -- Bloc anonyme de calcul et stockage
9  SET SERVEROUTPUT ON;
10 DECLARE
11     v_fact NUMBER;
12 BEGIN
13     FOR n IN 1..20 LOOP
14         v_fact := 1;
15         FOR i IN 1..n LOOP
16             v_fact := v_fact * i;
17         END LOOP;
18
19         INSERT INTO resultatsFactoriels (nombre, factorielle)
20         VALUES (n, v_fact);
21
22         DBMS_OUTPUT.PUT_LINE(n || '!' = ' ' || v_fact);
23     END LOOP;
24     COMMIT;
25 END;
26 /
27
28 -- Vérification
29 SELECT * FROM resultatsFactoriels ORDER BY nombre;
```

Listing 5 – Factorielles des 20 premiers entiers

Explications :

- La boucle externe itère de 1 à 20 pour chaque nombre.
- La boucle interne calcule la factorielle de chaque nombre.
- Chaque résultat est inséré dans la table `resultatsFactoriels`.
- Un seul `COMMIT` à la fin valide toutes les insertions.

Exercice n°2

Création de la table emp et insertion de données de test :

```
1 CREATE TABLE emp (  
2     matr      NUMBER(10)    NOT NULL,  
3     nom       VARCHAR2(50) NOT NULL,  
4     sal       NUMBER(7,2),  
5     adresse   VARCHAR2(96),  
6     dep       NUMBER(10)    NOT NULL,  
7     CONSTRAINT emp_pk PRIMARY KEY (matr)  
8 );  
9  
10 -- Insertion de données de test  
11 INSERT INTO emp VALUES (1, 'Ali', 3000,  
12     'rue de Paris', 92000);  
13 INSERT INTO emp VALUES (2, 'Sara', 3500,  
14     'avenue Victor Hugo', 75000);  
15 INSERT INTO emp VALUES (3, 'Mohamed', 2800,  
16     'boulevard Haussmann', 75000);  
17 INSERT INTO emp VALUES (5, 'Fatima', 4000,  
18     'rue de la Paix', 92000);  
19 INSERT INTO emp VALUES (6, 'Karim', 2200,  
20     'rue du Commerce', 10);  
21 INSERT INTO emp VALUES (7, 'Nadia', 3100,  
22     'place de la Nation', 10);  
23 COMMIT;
```

Listing 6 – Création de la table emp et données de test

Question 1 — Insertion d'un employé

Écrire un bloc anonyme qui permet d'insérer un nouveau employé (matr=4, nom=Youcef, sal=2500, adresse=avenue de la République, dep=92002).

```
1 SET SERVEROUTPUT ON;  
2 DECLARE  
3     v_employe emp%ROWTYPE;  
4 BEGIN  
5     v_employe.matr      := 4;  
6     v_employe.nom       := 'Youcef';  
7     v_employe.sal       := 2500;  
8     v_employe.adresse  := 'avenue de la Republique';  
9     v_employe.dep      := 92002;  
10  
11     -- Insertion du tuple dans la table  
12     INSERT INTO emp VALUES v_employe;  
13     COMMIT;  
14  
15     DBMS_OUTPUT.PUT_LINE('Employe insere avec succes.');
```

```
16 END;  
17 /
```

Listing 7 – Insertion d'un employé avec %ROWTYPE

Explications :

- emp%ROWTYPE déclare une variable de type enregistrement dont la structure correspond exactement à celle de la table emp.
- On affecte les valeurs à chaque champ de l'enregistrement avec l'opérateur :=.
- L'instruction INSERT INTO emp VALUES v_employe insère l'enregistrement complet dans la table.

Question 2 — Suppression avec SQL%ROWCOUNT

Écrire un bloc anonyme qui permet de supprimer tous les employés dont le dep est connu (ici dep = 10), et utiliser SQL%ROWCOUNT pour afficher le nombre de n-uplets supprimés.

```
1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_nb_lignes NUMBER;
4 BEGIN
5     DELETE FROM emp WHERE dep = 10;
6     v_nb_lignes := SQL%ROWCOUNT;
7
8     DBMS_OUTPUT.PUT_LINE('Nombre de lignes supprimees : '
9         || v_nb_lignes);
10    COMMIT;
11 END;
12 /
```

Listing 8 – Suppression et comptage avec SQL%ROWCOUNT

Explications :

- SQL%ROWCOUNT est un attribut implicite de curseur qui retourne le nombre de lignes affectées par la dernière instruction DML (INSERT, UPDATE, DELETE).
- Ici, il indique combien d'employés du département 10 ont été supprimés.

Question 3 — Somme des salaires avec curseur explicite

Écrire un bloc anonyme qui affiche la somme des salaires de tous les employés, en utilisant un curseur explicite et la boucle LOOP.

```
1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_salaire EMP.sal%TYPE;
4     v_total   EMP.sal%TYPE := 0;
5     CURSOR c_salaires IS
6         SELECT sal FROM emp;
7 BEGIN
8     OPEN c_salaires;
9
10    LOOP
```

```

11      FETCH c_salaires INTO v_salaire;
12      EXIT WHEN c_salaires%NOTFOUND;
13      IF v_salaire IS NOT NULL THEN
14          v_total := v_total + v_salaire;
15      END IF;
16  END LOOP;
17
18      CLOSE c_salaires;
19      DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
20 END;
21 /

```

Listing 9 – Somme des salaires — curseur explicite + LOOP

Explications :

- Un **curseur explicite** est déclaré avec `CURSOR c_salaires IS SELECT sal FROM emp.`
- On l'ouvre avec `OPEN`, on lit chaque ligne avec `FETCH ... INTO`, et on sort de la boucle lorsque `c_salaires%NOTFOUND` est vrai.
- On vérifie que le salaire n'est pas `NULL` avant de l'ajouter au total.
- On ferme le curseur avec `CLOSE`.

Question 4 — Salaire moyen avec curseur explicite

Modifier la procédure précédente pour calculer le salaire moyen.

```

1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_salaire    EMP.sal%TYPE;
4      v_total      EMP.sal%TYPE := 0;
5      v_compteur   NUMBER := 0;
6      v_moyenne    NUMBER(10,2);
7      CURSOR c_salaires IS
8          SELECT sal FROM emp;
9  BEGIN
10     OPEN c_salaires;
11
12     LOOP
13         FETCH c_salaires INTO v_salaire;
14         EXIT WHEN c_salaires%NOTFOUND;
15         IF v_salaire IS NOT NULL THEN
16             v_total := v_total + v_salaire;
17             v_compteur := v_compteur + 1;
18         END IF;
19     END LOOP;
20
21     CLOSE c_salaires;
22
23     IF v_compteur > 0 THEN
24         v_moyenne := v_total / v_compteur;
25         DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_moyenne);

```

```

26     ELSE
27         DBMS_OUTPUT.PUT_LINE('Aucun salaire trouve. ');
28     END IF;
29 END;
30 /

```

Listing 10 – Salaire moyen — curseur explicite + LOOP

Explications :

- On ajoute un compteur `v_compteur` qui s'incrémente pour chaque salaire non nul.
- La moyenne est calculée par la division `v_total / v_compteur`.
- On vérifie que le compteur est > 0 pour éviter une division par zéro.

Question 5 — Somme et moyenne avec boucle FOR IN

Modifier les deux procédures précédentes en utilisant la boucle `FOR IN`.

a) Somme des salaires avec FOR IN :

```

1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total EMP.sal%TYPE := 0;
4      CURSOR c_salaires IS
5          SELECT sal FROM emp;
6  BEGIN
7      FOR v_rec IN c_salaires LOOP
8          IF v_rec.sal IS NOT NULL THEN
9              v_total := v_total + v_rec.sal;
10         END IF;
11     END LOOP;
12
13     DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
14 END;
15 /

```

Listing 11 – Somme des salaires — boucle FOR IN

b) Salaire moyen avec FOR IN :

```

1  SET SERVEROUTPUT ON;
2  DECLARE
3      v_total      EMP.sal%TYPE := 0;
4      v_compteur   NUMBER := 0;
5      v_moyenne    NUMBER(10,2);
6      CURSOR c_salaires IS
7          SELECT sal FROM emp;
8  BEGIN
9      FOR v_rec IN c_salaires LOOP
10         IF v_rec.sal IS NOT NULL THEN
11             v_total := v_total + v_rec.sal;
12             v_compteur := v_compteur + 1;
13         END IF;

```



```

14     END LOOP;
15
16     IF v_compteur > 0 THEN
17         v_moyenne := v_total / v_compteur;
18         DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_moyenne);
19     ELSE
20         DBMS_OUTPUT.PUT_LINE('Aucun salaire trouve.');
```

Listing 12 – Salaire moyen — boucle FOR IN

Explications :

- La boucle FOR `v_rec IN c_salaires` LOOP simplifie le code : pas besoin de OPEN, FETCH, EXIT WHEN, ni CLOSE.
- Le curseur est automatiquement ouvert, parcouru et fermé.
- `v_rec` est une variable implicite de type enregistrement.

Question 6 — Curseur paramétré

Écrire une procédure anonyme qui affiche les noms des employés des départements 92000 et 75000, en utilisant un curseur paramétré.

```

1  SET SERVEROUTPUT ON;
2  DECLARE
3      CURSOR c(p_dep EMP.dep%TYPE) IS
4          SELECT dep, nom
5          FROM emp
6          WHERE dep = p_dep;
7  BEGIN
8      DBMS_OUTPUT.PUT_LINE('--- Departement 92000 ---');
9      FOR v_employe IN c(92000) LOOP
10         DBMS_OUTPUT.PUT_LINE(' ' || v_employe.nom);
11     END LOOP;
12
13     DBMS_OUTPUT.PUT_LINE('--- Departement 75000 ---');
14     FOR v_employe IN c(75000) LOOP
15         DBMS_OUTPUT.PUT_LINE(' ' || v_employe.nom);
16     END LOOP;
17 END;
18 /
```

Listing 13 – Curseur paramétré

Explications :

- Le curseur `c` accepte un paramètre `p_dep` de type `EMP.dep%TYPE`.
- On peut ainsi réutiliser le même curseur avec des valeurs différentes : `c(92000)` puis `c(75000)`.
- La boucle FOR IN gère automatiquement l'ouverture, le parcours et la fermeture du curseur.

Exercice n°3

Implémentation du package de gestion des clients avec deux procédures surchargées pour ajouter un client, et gestion des exceptions.

Création de la table client

```
1 CREATE TABLE client (  
2     id_client NUMBER(10) NOT NULL,  
3     nom       VARCHAR2(50) NOT NULL,  
4     prenom    VARCHAR2(50),  
5     adresse   VARCHAR2(100),  
6     telephone VARCHAR2(20),  
7     CONSTRAINT client_pk PRIMARY KEY (id_client)  
8 );
```

Listing 14 – Création de la table client

Spécification du package

```
1 CREATE OR REPLACE PACKAGE gestion_client AS  
2     -- Procédure 1 : ajout avec tous les attributs  
3     PROCEDURE ajouter_client(  
4         p_id          IN client.id_client%TYPE,  
5         p_nom         IN client.nom%TYPE,  
6         p_prenom      IN client.prenom%TYPE,  
7         p_adresse     IN client.adresse%TYPE,  
8         p_telephone   IN client.telephone%TYPE  
9     );  
10  
11     -- Procédure 2 : ajout avec seulement id et nom  
12     -- (surcharge : même nom, signature différente)  
13     PROCEDURE ajouter_client(  
14         p_id IN client.id_client%TYPE,  
15         p_nom IN client.nom%TYPE  
16     );  
17 END gestion_client;  
18 /
```

Listing 15 – Spécification du package gestion_client

Corps (body) du package

```
1 CREATE OR REPLACE PACKAGE BODY gestion_client AS  
2  
3     -- Procédure 1 : ajout complet  
4     PROCEDURE ajouter_client(  
5         p_id          IN client.id_client%TYPE,  
6         p_nom         IN client.nom%TYPE,  
7         p_prenom      IN client.prenom%TYPE,  
8         p_adresse     IN client.adresse%TYPE,
```

```

9      p_telephone IN client.telephone%TYPE
10 ) IS
11 BEGIN
12     INSERT INTO client (id_client, nom, prenom,
13                         adresse, telephone)
14     VALUES (p_id, p_nom, p_prenom,
15             p_adresse, p_telephone);
16     COMMIT;
17     DBMS_OUTPUT.PUT_LINE('Client ' || p_nom
18                          || ' ajoute avec succes (complet).');
19
20 EXCEPTION
21     WHEN DUP_VAL_ON_INDEX THEN
22         DBMS_OUTPUT.PUT_LINE('Erreur : le client avec id '
23                              || p_id || ' existe deja. ');
24         ROLLBACK;
25     WHEN OTHERS THEN
26         DBMS_OUTPUT.PUT_LINE('Erreur inattendue : '
27                              || SQLERRM);
28         ROLLBACK;
29 END ajouter_client;
30
31 -- Procédure 2 : ajout avec id et nom seulement
32 PROCEDURE ajouter_client(
33     p_id   IN client.id_client%TYPE,
34     p_nom  IN client.nom%TYPE
35 ) IS
36 BEGIN
37     INSERT INTO client (id_client, nom, prenom,
38                         adresse, telephone)
39     VALUES (p_id, p_nom, NULL, NULL, NULL);
40     COMMIT;
41     DBMS_OUTPUT.PUT_LINE('Client ' || p_nom
42                          || ' ajoute avec succes (simplifie).');
43
44 EXCEPTION
45     WHEN DUP_VAL_ON_INDEX THEN
46         DBMS_OUTPUT.PUT_LINE('Erreur : le client avec id '
47                              || p_id || ' existe deja. ');
48         ROLLBACK;
49     WHEN OTHERS THEN
50         DBMS_OUTPUT.PUT_LINE('Erreur inattendue : '
51                              || SQLERRM);
52         ROLLBACK;
53 END ajouter_client;
54
55 END gestion_client;
56 /

```

Listing 16 – Corps du package gestion_client avec exceptions

Tests du package

```
1  SET SERVEROUTPUT ON;
2
3  -- Test 1 : Ajout complet
4  BEGIN
5      gestion_client.ajouter_client(
6          1, 'Dupont', 'Jean',
7          '12 rue de la Paix', '0612345678');
8  END;
9  /
10
11 -- Test 2 : Ajout simplifié (surcharge)
12 BEGIN
13     gestion_client.ajouter_client(2, 'Martin');
14 END;
15 /
16
17 -- Test 3 : Tentative de doublon (exception DUP_VAL_ON_INDEX)
18 BEGIN
19     gestion_client.ajouter_client(1, 'Durand');
20 END;
21 /
22
23 -- Vérification
24 SELECT * FROM client;
```

Listing 17 – Tests du package gestion_client

Explications :

- La **surcharge** (*overloading*) est possible en PL/SQL : deux procédures portent le même nom `ajouter_client` mais avec des signatures différentes (nombre de paramètres différent).
- **Exceptions levées** :
 - `DUP_VAL_ON_INDEX` : levée lorsqu'on tente d'insérer un client dont l'`id_client` existe déjà (violation de la contrainte de clé primaire).
 - `OTHERS` : capture toute autre exception non prévue, avec affichage du message d'erreur via `SQLERRM`.
- En cas d'erreur, un `ROLLBACK` annule la transaction en cours.